

Algorithmique et fondements de l'informatique

INF109

Télécom Paris

Examen final du 26 janvier 2024

Sont autorisés : 2 feuilles A4 recto-verso manuscrites de taille lisible (4 pages)
& dictionnaire de traduction du français vers votre langue maternelle.
Sont interdits : matériel électronique, photocopie, notes de cours, livres et autres documents.

Les exercices sont indépendants et peuvent être traités dans n'importe quel ordre.
Dans chaque exercice, vous pouvez admettre la réponse à une question
pour résoudre les questions suivantes.

Lorsqu'un algorithme est attendu, sauf consigne particulière, vous pouvez en donner le principe, ou le présenter en pseudo-code, ou encore l'écrire en Python. Les erreurs de syntaxe Python ne sont pas comptées.

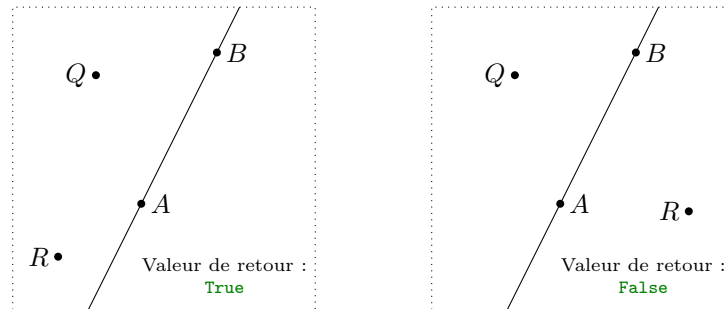
Durée : 3 heures. Barème : 180 points. Sujet en 1 pages.

Exercice 1. Dans cet exercice, nous nous appuyons sur la classe Python suivante qui permet de définir des points par leur abscisse et leur ordonnée et qui offre de calculer des différences ou des produits vectoriels.

```
1 class Point():
2     def __init__(self, abs, ord):
3         self.x = abs
4         self.y = ord
5     def difference(self, other):
6         Point(other.x - self.x, other.y - self.y)
7     def cross_prod(self, other):
8         return self.x * other.y - self.y * other.x
```

Nous ignorons les erreurs liées à des approximations numériques.

1. Proposer une fonction `same_side(Q, R, A, B)` qui vérifie que les deux points Q et R du plan appartiennent au même demi-plan fermé délimité par la droite (AB) . (7pts)



2. Proposer une fonction `in_triangle(Q, A, B, C)` qui vérifie que le point Q du plan appartient au triangle (ABC) , frontière incluse. (7pts)

Nous représentons tout polygone convexe $\mathcal{P}$ à n sommets $(A_0A_1 \cdots A_{n-1})$ par une liste Python dont les éléments appartiennent à la classe `Point`.

3. Proposer une fonction `in_polygon(Q, P)` de type « diviser pour régner » qui décide si un point Q du plan appartient au polygone convexe $\mathcal{P}$. (9pts)
4. Déterminer la complexité en temps de la fonction `in_polygon` en fonction de n . (4pts)
5. La fonction `in_polygon` proposée fonctionne-t-elle également si le polygone n'est pas convexe ? (5pts)

Solution 1. L'un des exercices données en devoir à la maison portait déjà sur un problème de géométrie. Cet exercice permettait de valoriser les élèves qui l'avaient travaillé et s'étaient familiarisé avec l'utilisation du produit vectoriel pour résoudre certaines questions. De nombreuses copies ont voulu s'en tirer en passant par des mises en équations (de la forme $y = \alpha x + \beta$) ce qui était une très mauvaise idée : le code est long, compliqué et souvent incomplet puisque le cas de droites verticales doit alors être traité à part. Dans la question 3, peu de copies connaissent la dichotomie ou l'appliquent avec beaucoup de mauvaise foi.

1. Le lieu des points M appartenant à l'un des demi-plans fermés délimité par la droite (AB) est déterminé par l'un des deux équations $\overrightarrow{AB} \wedge \overrightarrow{AM} \geq 0$ (demi plan situé à gauche pour un observateur placé en A et regardant vers B) et $\overrightarrow{AB} \wedge \overrightarrow{AM} \leq 0$ (demi plan situé à droite pour un observateur placé en A et regardant vers B). Deux points Q et R sont du même côté de (AB) si et seulement si les produits vectoriels $\overrightarrow{AB} \wedge \overrightarrow{AQ}$ et $\overrightarrow{AB} \wedge \overrightarrow{AR}$ sont de même signe (au sens large), ce qui se vérifie en testant

$$(\overrightarrow{AB} \wedge \overrightarrow{AQ}) \cdot (\overrightarrow{AB} \wedge \overrightarrow{AR}) \geq 0.$$

On écrit en conséquence le code suivant :

```

9  def same_side(Q, R, A, B):
10     AB = A.diff(B)
11     AQ = A.diff(Q)
12     AR = A.diff(R)
13     z1 = AB.cross_prod(AQ)
14     z2 = AB.cross_prod(AR)
15     return(z1*z2 >= 0)

```

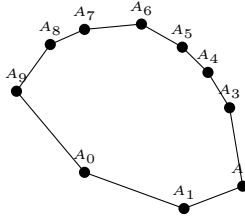
2. Un point Q est à l'intérieur d'un triangle (ABC) si et seulement si les points Q et A sont du même côté par rapport à la droite (BC) , les points Q et B sont du même côté par rapport à la droite (AC) et enfin les points Q et C sont du même côté par rapport à la droite (AB) .
Il vient donc le code suivant.

```

16 def in_triangle(P, A, B, C):
17     return( same_side(P, A, B, C) and
18             same_side(P, B, A, C) and
19             same_side(P, C, A, B))

```

3. Soit $\mathcal{P}$ un polygone convexe à n sommets $(A_0A_1A_2\cdots A_{n-1})$. Nous commençons par vérifier que les points sont énumérés dans l'ordre trigonométrique : s'il existe un indice i tel que $\overrightarrow{A_iA_{i+1}} \wedge \overrightarrow{A_{i+1}A_{i+2}} < 0$ (où A_n s'entend comme le point A_0), nous remplaçons la liste des points par $(A_{n-1}\cdots A_1A_0)$. Quitte à utiliser les points dans l'ordre inverse, on peut supposer que les points sont placés dans l'ordre trigonométrique.



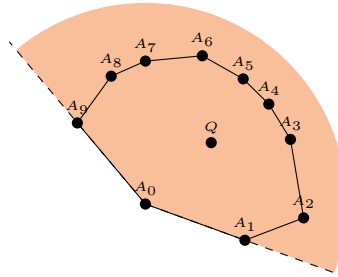
Nous écrivons deux prédicats logiques qui permettent de tester si un point Q se trouve à gauche ou à droite de la droite (AB) (orientée de A vers B). Cela revient simplement à tester le signe du produit vectoriel $\overrightarrow{AB} \wedge \overrightarrow{AQ}$.

```

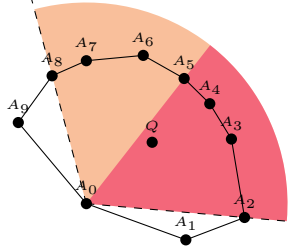
20 def is_left(Q, A, B):
21     AB = A.diff(B)
22     AQ = A.diff(Q)
23     return(AB.cross_prod(AQ) >= 0 )
24
25 def is_right(Q, A, B):
26     AB = A.diff(B)
27     AQ = A.diff(Q)
28     return(AB.cross_prod(AQ) <= 0 )

```

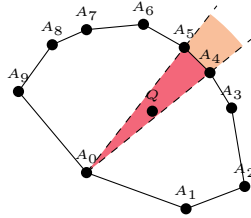
Nous commençons par nous assurer que Q est dans le secteur angulaire $\widehat{A_1A_0A_{n-1}}$, c'est-à-dire Q est à gauche de (A_0A_1) et à droite (A_0A_{n-1}) .



Puis, par dichotomie, nous restreignons ce secteur angulaire et nous recherchons deux points contigus A_i et A_{i+1} tels que Q est à gauche de (A_0A_i) et à droite (A_0A_{i+1}) .



Enfin, nous nous assurons que Q est à l'intérieur du triangle $(A_0A_iA_{i+1})$.



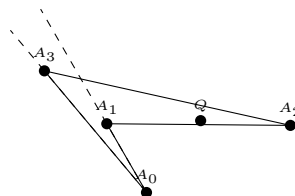
Nous proposons le code suivant

```

29 def in_polygon(Q, P):
30     if not (is_left(Q, P[0], P[1]) and
31             is_right(Q, P[0], P[-1])):
32         return False
33     else:
34         left = len(P)-1
35         right = 1
36         # Dichotomie pour localiser le secteur
37         while left-right > 1:
38             mid = (right - left)/2
39             if is_right(Q, P[0], P[mid]):
40                 right = mid
41             else:
42                 left = mid
43         # Test final
44         return in_triangle(Q, P[0], P[left], P[right])

```

4. Chacun des appels à des prédicats tels que `is_left`, `is_right` ou `in_triangle` se fait en temps constant. Concernant l'analyse de la dichotomie, nous nous appuyons sur la quantité v valant `right - left` qui est un *variant de boucle*. Initialement, le variant v vaut $n - 2$; puis, le variant v est divisé par deux à chaque étape. La boucle se termine quand le variant devient inférieur à 1 : il faut donc $O(\log_2 n)$ étapes pour terminer la boucle `while`. Au total, la complexité de `in_polygone` est $O(\log_2 n)$.
5. Dans le cas où le polygone n'est pas convexe, la fonction proposée n'a plus de sens. Prenons l'exemple



Le premier secteur est entre les droites A_0A_1 et A_0A_3 : il exclut la zone où se trouve le point Q .

Exercice 2. Le problème SOMMEQUATERNAIRE est le problème de décision suivant : « étant donné un tableau de n entiers relatifs ($n \geq 1$), existe-t-il quatre d'entre eux dont la somme est nulle ? ».

1. Nous proposons le code suivant

```

45 def decide_4sum(T):
46     for a in T:
47         for b in T:
48             for c in T:
49                 for d in T:
50                     if a + b + c + d == 0:
51                         return True
52     return False

```

Déterminer la complexité en temps et la complexité en mémoire de la fonction `decide_4sum` en fonction de n . (4pts)

2. Proposer une fonction `better_decide_4sum` qui décide le problème SOMMEQUATERNAIRE avec une complexité en temps $O(n^2)$ et une complexité en mémoire $O(n^2)$. (16pts)

On décrira avec soin le (ou les) type abstrait employé ainsi que la (ou les) structure de données le réalisant ; on indiquera précisément les opérations fournies par le type et les complexités de leur implémentation par la structure concrète.

Solution 2. 1. Le code proposé enchaîne quatre boucles qui s'appliquent à n termes. Le calcul de la somme et le test à 0 se fait en temps constant. La complexité en temps est donc $O(n^4)$.

Le parcours de la liste pour les valeurs de a , b , c et d (lignes 46-49) consomme un temps constant (il faut simplement se souvenir où est positionné l'itérateur). Il en va de même pour le calcul de la somme $a+b+c+d$ (en ligne 50) qui se fait dans une variable locale au programme. Le programme consomme donc un espace $O(1)$ en mémoire. La complexité en espace est donc $O(1)$. Certaines copies comptaient aussi l'espace requis pour écrire les données d'entrée (la liste T), ce qui était aussi acceptable, même si, au sens strict, le progr.

2. Soient a , b , c et d trois entiers dont la somme est nulle. On a alors l'équation

$$a + b = -(c + d).$$

Ceci nous suggère la stratégie suivante. Nous calculons l'ensemble E des sommes de deux éléments de T . Si l'entier 0 apparaît dans E , nous pouvons

résoudre le problème de décision en utilisant la somme quaternaire $a + b + a + b = 0$. Sinon, il nous suffit de tester si E contient une valeur et son opposé.

Nous nous proposons d'utiliser une collection correspondant au type abstrait « ensemble » qui puisse contenir des entiers relatifs. Nous avons besoin de trois opérations :

- (a) la création d'un ensemble vide, $() \rightarrow \text{Ensemble}$,
- (b) l'insertion d'un entier, $\text{Ensemble}, \text{Entier} \rightarrow \text{Ensemble}$, et
- (c) la consultation, $\text{Ensemble}, \text{Entier} \rightarrow \text{Booléen}$, c'est-à-dire le test de présence d'un élément dans la table.

Nous avons vu en cours que le type abstrait peut se réaliser avec une table de hachage. Comme nous allons placer $n(n+1)/2$ éléments dans le pire des cas, afin de garantir que le bon fonctionnement de la table, nous pouvons créer d'emblée un tableau de capacité $\Theta(n^2)$, par exemple en fixant la capacité à $2n^2$: nous sommes alors certain que le facteur de charge reste inférieure à 25% dans le pire des cas.

La création de la table, l'insertion et le test de présence d'un élément dans la table se font en temps $O(1)$ pour chaque opération.

Nous proposons finalement le code suivant :

```

53 def better_decide_4sum(T):
54     E = set()
55     n = len(T)
56     for i in range(n):
57         for j in range(i, n):
58             s = T[i] + T[j]
59             if s == 0:
60                 return True
61             elif -s in E:
62                 return True
63             else:
64                 E.add(s)
65     return False

```

Certaines copies ont proposé de créer la liste

$$L = [a + b \text{ for } a \text{ in } T \text{ for } b \text{ in } T]$$

puis de tester les sommes de couples dans L :

$$\text{any}([x + y == 0 \text{ for } x \text{ in } L \text{ for } y \text{ in } L]).$$

Cette solution ne fonctionne pas, puisqu'elle conduit à parcourir deux fois de façon imbriquée la liste L qui est une liste de n^2 éléments : la complexité finale est donc $O((n^2)^2)$ ce qui reste trop élevé.

Exercice 3. Dans cet exercice, les arbres binaires de recherche peuvent posséder plusieurs fois la même valeur comme étiquette. Par ailleurs, les sommets des arbres binaires étiquetés sont représentés par la classe Python suivante

```

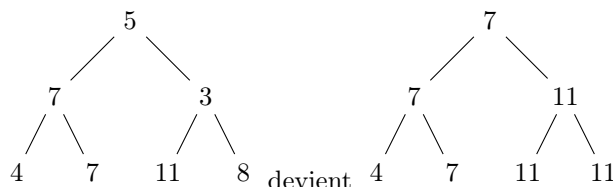
66 class Node():
67     def __init__(self, d, a = None, b = None):
68         self.data = d
69         self.left = a
70         self.right = b

```

Nous nous intéressons au problème suivant. Étant donné un arbre binaire arbitraire étiqueté par des entiers, que nous notons A , nous cherchons un nouvel arbre binaire étiqueté A' tel que

- (i) l'arbre A' a la même forme que l'arbre A
- (ii) si e' est l'étiquette d'un sommet dans l'arbre A' et si e est l'étiquette du sommet à la même position dans l'arbre A , alors nous avons $e' \geq e$.
- (iii) l'arbre A' est un arbre binaire de recherche
- (iv) la somme des poids des étiquettes de A' est aussi petite que possible.

Exemple :



1. Rappeler la complexité en temps d'un parcours d'arbre. (2pts)
2. Montrer que l'on peut adapter l'un des algorithmes de parcours d'arbre afin de résoudre le problème :
 - (a) Donner le nom du parcours d'arbre utilisé. (2pts)
 - (b) Décrire, sous la forme d'un pseudo-code ou d'une fonction python, un algorithme de transformation de l'arbre A (dans cette question, l'arbre A est modifié en l'arbre A'). (10pts)
 - (c) Décrire, sous la forme d'un pseudo-code ou d'une fonction python, un algorithme de construction de l'arbre A' (dans cette question, l'arbre A doit rester intact). (10pts)

Solution 3. Cet exercice est subtil puisqu'il demandait de bien comprendre la nuance entre structure de donnée mutable et structure de donnée immuable. Pour comprendre le corrigé, il est très fortement recommandé d'exécuter le code dans <https://pythontutor.com/> pour comprendre ce qui se passe précisément avec les objets en mémoire.

1. La complexité en temps d'un parcours d'arbre se fait en temps linéaire par rapport au nombre de sommets dans l'arbre.
2. (a) Un arbre binaire de recherche est caractérisé par le fait que, quand on parcourt ses éléments en ordre infixe (ou parcours symétrique), la suite des étiquettes lues est la suite des éléments de l'arbre triée par ordre croissant.

Le problème ici est donc d'augmenter chaque étiquette, d'une valeur aussi faible que possible, de sorte qu'en ordre infixe, les étiquettes rencontrées soient croissantes. Autrement dit, si nous notons

$[e_1, e_2, \dots, e_n]$ les étiquettes de l'arbre initial dans l'ordre infixe et $[e'_1, e'_2, \dots, e'_n]$ après transformation, nous devons avoir pour tout indice i compris entre 1 et n

$$\begin{cases} e_i & \leq e'_i \\ \max_{1 \leq j \leq i-1} e'_j & \leq e'_i \end{cases}$$

et nous cherchons à minimiser la somme $\sum_{i=1}^n (e'_i - e_i)$, ce qui revient simplement à minimiser $\sum_{i=1}^n e'_i$. Nous constatons que la valeur e'_i ne dépend que de données de l'énoncé et des valeurs e'_j pour $j < i$. En conséquence, nous pouvons résoudre ce problème de manière gloutonne fixant la valeur de e'_1 (égale à e_1), puis de e'_2 (égale à $\min(e_2, e'_1)$), puis etc. e'_i (égale à $\min(e_i, e'_{i-1})$). La résolution peut se faire directement en parcourant l'arbre en maintenant une variable qui indique la plus grande valeur rencontrée, c'est-à-dire la valeur de la dernière étiquette.

- (b) Nous commençons par écrire une fonction auxiliaire qui a pour effet de transformer l'arbre passé en paramètre et dont la valeur de retour est le maximum déjà rencontré.

```

71 def transform_with_bound(node, bound):
72     if node.left is not None:
73         bound = transform_with_bound(node.left, bound)
74     bound = max(bound, node.data)
75     node.data = bound
76     if node.right is not None:
77         bound = transform_with_bound(node.right, bound)
78     return bound

```

L'énoncé ne le précisant pas, nous faisons l'hypothèse que les entiers sont des entiers naturels et que 0 est le plus petit d'entre eux. Nous écrivons finalement le code suivant qui a pour effet de transformer l'arbre passé en paramètre et qui n'a aucune valeur de retour.

```

79 def transform(node):
80     transform_with_bound(node, 0)

```

Si des entiers négatifs sont autorisés, il faut remplacer 0 par l'entier rencontré en descendant le plus à gauche, que l'on calcule comme suit.

```

81 def left_most(node):
82     if node.left is not None:
83         return left_most(node.left)
84     else:
85         return node.data

```

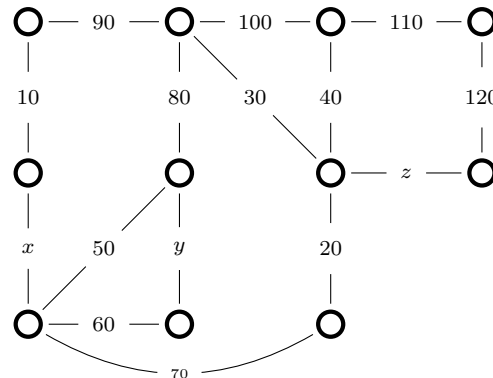
- (c) Pour répondre à cette question, nous reprenons les éléments de la question précédente et nous introduisons des constructeurs :


```

86 def construct_with_bound(node, bound):
87     if node.left is not None:
88         newleft, bound = construct_with_bound(node.left,
89         ↪ bound)
89     else:
90         newleft = None
91     bound = max(bound, node.data)
92     newdata = bound
93     if node.right is not None:
94         newright, bound =
95         ↪ construct_with_bound(node.right, bound)
96     else:
97         newright = None
98     return Node(newdata, newleft, newright), bound
99
100 def construct(node):
101     newnode, _ = construct_with_bound(node, 0)
102     return newnode

```

Exercice 4. Soient x , y et z trois entiers naturels. Nous considérons le graphe pondéré suivant :



Nous supposons qu'un certain arbre couvrant de poids minimum contient les arêtes de poids x , y et z .

1. Dresser la liste des poids des arêtes autres que x , y et z appartenant à cet arbre couvrant de poids minimum. Comment est-il possible de calculer un tel arbre ? (5pts)
2. Est-il possible que la valeur de x soit 120 ? (2pts)
3. Est-il possible que la valeur de y soit 55 ? (2pts)
4. En supposant que les arêtes ont toutes des poids distincts, quelle est la plus grande valeur possible de z ? (3pts)

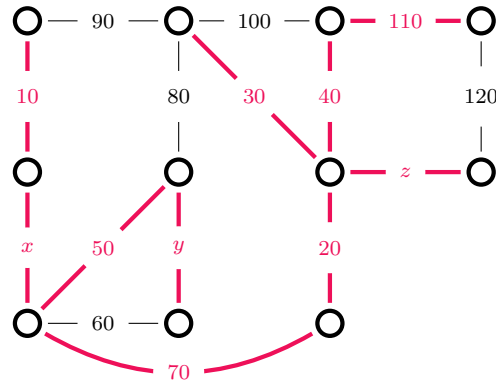
Solution 4. 1. Pour obtenir un arbre contenant les arêtes de poids x , y et z , on peut appliquer l'algorithme de Kruskal avec une initialisation dans laquelle les trois arêtes sont déjà sélectionnées. (Alternativement, on peut

aussi fixer temporairement $x = y = z = 1$: comme le poids 1 est plus petit que le poids des arêtes existantes, avec l'algorithme de Kruskal, on est assuré d'obtenir un arbre couvrant qui contient x , y et z).

Si nous appliquons l'algorithme de Kruskal, nous sommes amenés à sélectionner les arêtes de poids

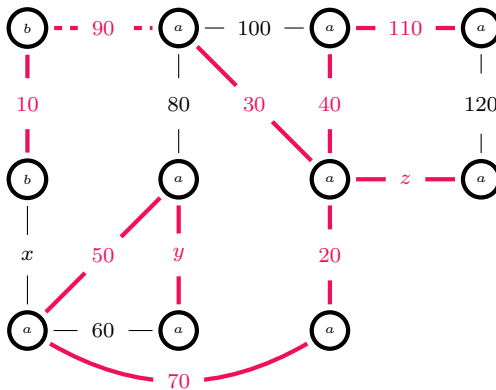
$$x \quad y \quad z \quad 10 \quad 20 \quad 30 \quad 40 \quad 50 \quad 70 \quad 110.$$

et finalement l'arbre est l'arbre A_1 suivant :



(Dans cette question, il y a eu souvent des fautes d'étourderie : par exemple sélectionner l'arête de poids 80 au lieu de celle de poids 70.)

- Supposons qu'il existe un arbre A_2 de poids minimum contenant l'arête x , dans le cas où celle-ci est de poids 120 et notons a et b les deux extrémités de cette arête. Si nous supprimons l'arête de poids x , nous obtenons deux composantes connexes : celle du sommet a et celle du sommet b . Indiquons à quelle composante appartiennent les sommets le long du cycle $x - 10 - 90 - 80 - 50$: il doit exister au moins une autre arête x' distincte de x reliant les deux composantes. En considérant $A_3 = A_2 \setminus \{x\} \cup \{x'\}$, nous avons construit un nouvel arbre de poids total plus petit.

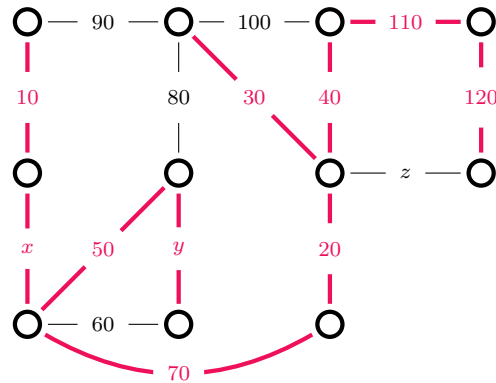


- L'arête y peut prendre la valeur 55. Le déroulement de l'algorithme de Kruskal serait dans ce cas :

$$x \quad z \quad 10 \quad 20 \quad 30 \quad 40 \quad 50 \quad y \quad 80 \quad 110.$$

On retombe sur l'arbre A_1 de la première question.

4. Pour toute valeur entière $z < 120$, l'arbre construit par l'algorithme de Kruskal est l'arbre A_1 dessiné à la question 1. Pour tout $z > 120$, l'algorithme de Kruskal va privilégier l'arête de poids 120 et produire un autre arbre A_4



La valeur maximum est donc $z = 119$.

Remarque : si $z = 120$, les deux arbres A_1 et A_4 sont possibles puisqu'ils ont le même poids total.

Remarque générale. Pour répondre correctement à cet exercice, il faut être précis dans les quantificateurs sous-jacents. En effet, il peut exister plusieurs arbres couvrants de poids minimal : il ne suffit donc pas de parler de l'« arbre créé par l'algorithme de Kruskal » d'autant plus que celui-ci n'est pas forcément parfaitement défini quand deux arêtes ont le même poids. Pour démontrer proprement la question 2, il faut exhiber un arbre différent de poids strictement plus petit et non pas se contenter de dire que l'algorithme de Kruskal ne rend pas un arbre utilisant l'arête de poids x .

Exercice 5. Le satrape *Lysimaque*, fraîchement nommé dans une province lointaine, a reçu pour mission d'assurer l'ordre et la paix sur un territoire récemment conquis. Il projette de construire un ensemble d'arènes dans quelques-unes des villes principales de sa satrapie ainsi que de faire paver un ensemble de routes de sorte que tout notable, qui réside forcément en ville, puisse accéder à l'une des arènes à partir de sa résidence. Un vieil adage ne dit-il pas que tant que le peuple et leurs meneurs sont occupés aux jeux, ils ne pensent pas à se rebeller ?

Il faut savoir que les notables ne se déplacent qu'en char. Aussi, un notable ne peut atteindre que des villes reliées à sa ville d'origine par des routes pavées, en passant éventuellement par des villes intermédiaires. La guerre a détruit de nombreuses infrastructures : il ne reste aucune arène ni route entière. Le satrape a fait établir par ses ingénieurs le coût de construction ou de reconstruction d'une arène ville par ville ainsi que le coût de pavage de chaque route entre chaque paire de villes pouvant être reliées. Lysimaque possède un budget limité et souhaite dépenser le moins de drachmes possible.

1. Exprimer la situation énoncée ci-dessus sous la forme d'un problème II (5pts)
mathématiquement spécifié. On donnera une description des instances

d'entrées, de la valeur de retour, d'éventuelles préconditions ou post-conditions.

2. Réduire le problème Π à un problème classique Γ étudié en classe. (14pts)
3. Nommer un algorithme classique permettant de résoudre Γ . (4pts)
4. Calculer une solution dans le cas où les coûts sont les suivants (7pts)

Ville	Coût de l'arène
Arbalès	7
Aspardana	3
Babylone	4
Rhagai	6
Suse	8

Route	Coût du pavage
Arbalès – Aspardana	6
Arbalès – Babylone	1
Arbalès – Rhagai	9
Aspardana – Babylone	3
Babylone – Suse	7
Rhagai – Suse	1

Il est demandé de détailler les étapes proposées dans les questions qui précèdent et de donner des détails concernant le déroulement de l'algorithme proposé à la question 3.

Solution 5. 1. Le problème Π peut être formalisé comme le problème d'optimisation suivant :

- Données d'entrée : un graphe non orienté $G = (V, E)$, une application $\alpha : V \rightarrow \mathbb{N}$ pondérant les sommets, une application $\delta : E \rightarrow \mathbb{N}$ pondérant les arêtes.
- Valeur de retour : ensemble de sommets $A \subseteq V$ et ensemble d'arêtes $R \subseteq E$
- Préconditions : aucune
- Postconditions :
 - (a) Pour tout sommet $v \in V$, il existe un sommet $a \in A$ accessible depuis v dans le graphe (V, R) .
 - (b) La somme

$$\sum_{a \in A} \alpha(a) + \sum_{r \in R} \delta(r)$$

est aussi petite que possible.

Remarque 1 : Certaines copies ont proposé des fonctions α et δ à valeur quelconque et ont indiqué la positivité sous forme d'une précondition.

Remarque 2 : Un nombre étonnamment élevé de copies a cherché à répondre à cette question sans prononcer le mot « graphe ». Nous redisons que l'objectif d'une modélisation *mathématique* est de s'extraire des éléments concrets d'une situation particulière (ici, les villes, les arènes, les routes) et de les rapporter à des objets mathématiques génériques (ici le graphe valué) sur lesquels on peut appliquer des méthodes elles-aussi génériques (ici des algorithmes de graphe).

2. Nous nous proposons de réduire le problème II au problème d'optimisation Γ de recherche d'un *arbre couvrant de poids minimum* qui a été rencontré en cours.

Soit $(G = (V, E), \alpha, \rho)$ une instance du problème II. Nous construisons un nouveau graphe $\hat{G}$ ayant pour ensemble de sommet

$$\hat{V} = V \cup \{s\}$$

où s est un sommet frais, qui n'existait pas dans V , et pour ensemble d'arête

$$\hat{E} = E \cup \{(s, v); v \in V\}.$$

Nous construisons une pondération ρ de $\hat{E}$ comme suit :

$$\forall e \in \hat{E}, \quad \rho(e) = \begin{cases} \alpha(v) & \text{si } e \text{ est de la forme } e = (s, v) \text{ avec } v \in V \\ \delta(e) & \text{si } e \text{ appartient à } E \end{cases}$$

Il est clair que $\hat{G} = (\hat{V}, \hat{E}, \rho)$ est une instance du problème Γ

Soit $T \subseteq \hat{E}$ un arbre couvrant optimum de $\hat{G}$. Montrons que nous pouvons construire une solution relative à l'instance du problème initial II. Nous appelons $A = \{v \in V \text{ t.q. } (s, v) \in T\}$; nous appelons $R = T \cap E$.

Il nous faut voir que les postconditions (a) et (b) sont bien satisfaites :

- (a) Soit v un sommet de V . A-t-on bien une route vers un sommet de A ?

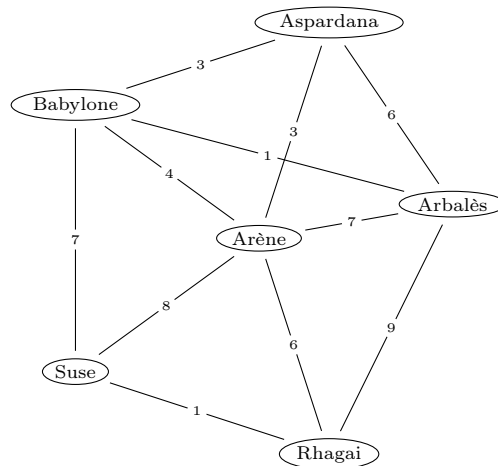
Comme T est un arbre, il existe un unique chemin élémentaire allant de v à s . Notons a l'avant dernier sommet sur ce chemin. On a donc aussi un chemin de v vers a , qui ne passe pas par s , c'est-à-dire un chemin de v vers a qui passe uniquement par des arêtes de E . Notons que a appartient à A ce qui achève la preuve.

- (b) La construction décrite ci-dessus conserve les poids et est réversible : à partir d'un ensemble de sommets A et d'un ensemble d'arête R , nous pouvons obtenir un sous-graphe couvrant de $\hat{G}$ de même poids. De plus, si le poids est minimum, il s'agit forcément d'un arbre.

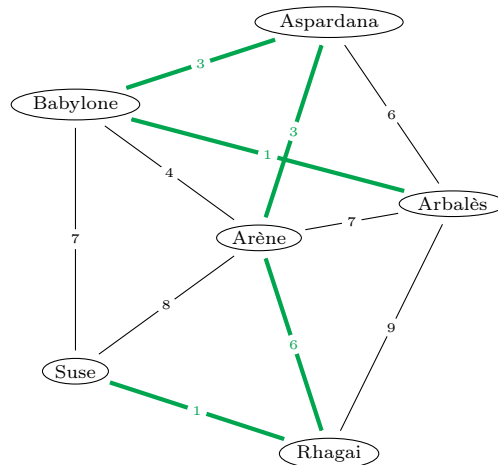
Nous avons ainsi réduit le problème II au problème Γ .

Remarque : Certaines copies ont pensé que la solution optimale n'utilisait forcément qu'une seule arête. Elles proposaient la solution suivant : déterminer l'arête de plus bas prix d'une part, relier toutes les villes entre elle par un arbre couvrant d'autre part. Cette solution est fausse comme le montre l'exemple de la question 4.

3. Il y a deux algorithmes classiques pour résoudre le problème Γ : l'algorithme de Kruskal ou l'algorithme de Prim.
4. Dans le cas présent, nous obtenons le graphe pondéré dessiné comme suit :



ce qui donne l'arbre couvrant suivant



Nous en déduisons le projet de travaux suivants :

- construire deux arènes (l'une à Aspardana, l'autre à Rhagai)
- et construire les routes suivantes : Aspardana – Babylone – Arbalès et Rhagai – Suse.

Ces travaux coûtent 14 drachmes.

Exercice 6. Dans cet exercice, nous posons $\Sigma = \{u, v, w, x, y, z\}$. Nous souhaitons représenter le mot

$s = vvvvuxxxuzzxvxxvxyxxvuzvvzyvxxwxuyuvyyvxxvuzzxuxx$

par un mot binaire. Nous avons compté le nombre d'occurrences de chaque symbole :

Symbole	u	v	w	x	y	z
Occurences	7	12	2	16	6	7

1. Exhiber un code binaire $\Sigma \rightarrow \{0, 1\}^*$ uniquement décodable à longueur fixe tel que la longueur de codage du mot s soit 150 bits. (3pts)

2. Déterminer un arbre de compression optimal pour le mot s en appliquant l'algorithme de Huffman et en indiquant les étapes de son déroulement. (6pts)
3. Calculer la longueur de codage du mot s avec le code de la question 2. (3pts)

Solution 6. 1. Comme il y a 6 symboles à coder et comme $4 = 2^2 < 6 \leq 2^3 = 8$, on peut utiliser un code à longueur fixe sur 3 bits. Par exemple,

Symbole	u	v	w	x	y	z
Codon	000	001	010	011	100	101

Nous rappelons que ce code à longueur fixe est bien entendu uniquement décodable, puisqu'on peut décoder paquet de trois bits par paquets de trois bits.

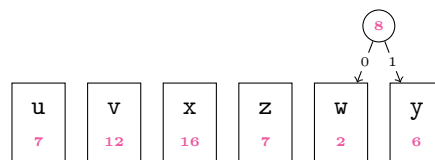
Comme le mot s est de longueur $|s| = 50$, sa longueur de codage vaut $3 \cdot 50 = 150$ bits.

2. Nous démarrons l'algorithme de Huffman en initialisant une file à priorité qui contient les arbres suivants :

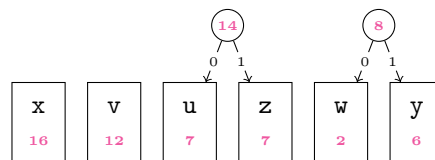


(Nos représentations graphiques ne sont pas ordonnées selon un ordre particulier : le seul point important est que nous savons retirer l'élément de plus petite priorité.)

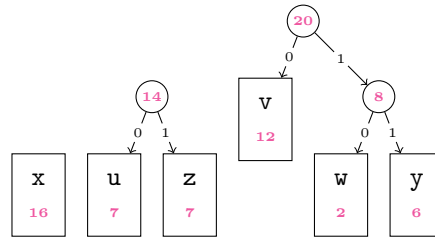
Nous tirons de la file à priorité les deux éléments de plus bas score, qui sont 2 et 6. Nous les combinons en un nouvel arbre dont le poids est 8. Nous réinsérons cet arbre dans la file, qui contient désormais les éléments suivants.



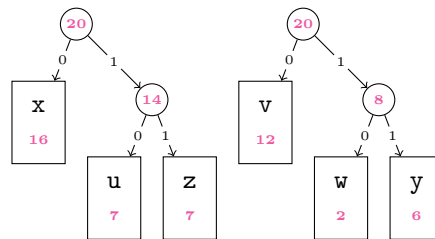
Nous tirons de la file à priorité les deux éléments de plus bas score, qui sont 7 et 7. Nous les combinons en un nouvel arbre dont le poids est 14. Nous réinsérons cet arbre dans la file, qui contient désormais les éléments suivants.



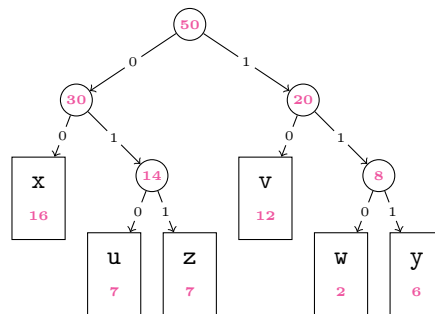
Nous tirons de la file à priorité les deux éléments de plus bas score, qui sont 8 et 12. Nous les combinons en un nouvel arbre dont le poids est 20. Nous réinsérons cet arbre dans la file, qui contient désormais les éléments suivants.



Nous tirons de la file à priorité les deux éléments de plus bas score, qui sont 14 et 16. Nous les combinons en un nouvel arbre dont le poids est 30. Nous réinsérons cet arbre dans la file, qui contient désormais les éléments suivants.



Finalement nous joignons les deux arbres restants. Nous obtenons l'arbre suivant.



Nous avons obtenu le code binaire

Symbole	u	v	w	x	y	z
Codon	010	10	110	00	111	011

D'autres solutions étaient possible, selon la manière de combiner les arbres à chaque étape.

3. Avec ce nouveau code, la longueur de codage de s vaut

$$7 \cdot 3 + 12 \cdot 2 + 2 \cdot 3 + 16 \cdot 2 + 6 \cdot 3 + 7 \cdot 3 = 50 + 30 + 20 + 14 + 8$$

soit 122 bits. (Même si votre arbre prend une forme différente, vous devez tomber sur ce chiffre à la fin)

Exercice 7. Dans cet exercice, nous posons $\Sigma = \{\mathbf{a}, \mathbf{b}\}$. Nous nous intéressons au langage rationnel $L(r)$ dénoté par l'expression régulière $r = \mathbf{ab}^*\mathbf{a|aba}^*$.

1. Dire quels sont tous les mots de longueur ≤ 3 qui appartiennent au langage $L(r)$. (3pts)
2. Dessiner un arbre de dérivation qui justifie que r est une expression régulière. (5pts)
3. Construire l'automate de Glushkov relatif à l'expression régulière $r_1 = ab*a$. Il est demandé de noter 0, 1, 2 et 3 les états successivement rencontrés dans l'automate lorsqu'on consomme les lettres du mot particulier aba et d'utiliser d'autres entiers pour les états restants. (6pts)

Nous appelons $\mathcal{A}_1$ l'automate résultant.

4. Construire l'automate de Glushkov relatif à l'expression régulière $r_2 = aba*$. Il est demandé de noter 0', 1', 2' et 3' les états successivement rencontrés dans l'automate lorsqu'on consomme les lettres du mot particulier aba et d'utiliser d'autres entiers pour les états restants. (6pts)

Nous appelons $\mathcal{A}_2$ l'automate résultant. Nous appelons enfin $\mathcal{A}$ l'automate obtenu en juxtaposant simplement $\mathcal{A}_1$ et $\mathcal{A}_2$ (*i.e.* le dessin de $\mathcal{A}$ s'obtient en dessinant $\mathcal{A}_1$ et $\mathcal{A}_2$ côte à côte sans rien ajouter ou modifier).

5. Citer une relation entre le nombre d'états d'un automate obtenu par l'algorithme de Glushkov et une expression régulière. (2pts)
6. Dire si l'automate $\mathcal{A}$ coïncide avec l'automate de Glushkov relatif à l'expression régulière r . (2pts)
7. Appliquer l'algorithme de détermination à l'automate $\mathcal{A}$. Il est demandé de suivre scrupuleusement la procédure vue en classe et d'indiquer suffisamment d'étapes de calcul pour en comprendre la construction. On vérifiera son résultat avec les éléments de la question 1 ; on s'assurera que l'automate obtenu est bien complet. (10pts)

Nous appelons $\mathcal{D}$ l'automate déterministe complet (DFA) résultant.

8. Nommer, s'ils existent, les états *puits* de l'automate $\mathcal{D}$. (2pts)
9. Appliquer l'algorithme de minimisation à l'automate $\mathcal{D}$. Il est demandé de suivre scrupuleusement la procédure vue en classe avec suffisamment d'étapes pour en comprendre la construction. On vérifiera son résultat avec les éléments de la question 1. (10pts)

Nous appelons $\mathcal{M}$ l'automate déterministe complet (DFA) résultant.

10. Que peut-on dire de la longueur de l'expression régulière obtenue en appliquant l'algorithme d'élimination des états à l'automate $\mathcal{M}$ par rapport à la longueur de r ? (4pts)

Solution 7. 1. Il n'y a pas de mots de longueur 0 ou 1. Les mots de longueur 2 sont aa et ab . Il y a un mot de longueur 3 qui est aba .

2. Nous écrivons l'arbre de dérivation suivant, dans lequel nous omettons d'ajouter la précision $\in \text{RegExp}(\Sigma)$ (que nous pourrions rendre présente dans chaque conclusion)

$$\begin{array}{c}
 \frac{}{a} \text{ (Lt.)} \quad \frac{\frac{b}{b*} \text{ (Ét.)}}{ab*} \text{ (Con.)} \quad \frac{}{a} \text{ (Lt.)} \quad \frac{\frac{a}{a} \text{ (Lt.)} \quad \frac{b}{b} \text{ (Lt.)}}{ab} \text{ (Con.)} \quad \frac{\frac{a}{a*} \text{ (Ét.)}}{aba*} \text{ (Con.)} \\
 \hline
 \frac{ab*a \quad ab*a*}{ab*a | aba*} \text{ (Un.)}
 \end{array}$$

Cet arbre n'est pas le seul possible. Nous avons choisi de commencer par s'occuper d'abord des dérivations à gauche.

Une autre réponse possible serait par exemple

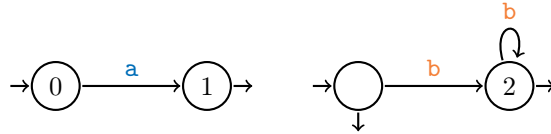
$$\begin{array}{c}
 \frac{\overline{a} \text{ (Lt.)}}{\frac{\overline{b} \text{ (Lt.)}}{\frac{\overline{b^*} \text{ (Ét.)}}{\frac{\overline{a} \text{ (Lt.)}}{\frac{\overline{b^*a} \text{ (Con.)}}{\overline{ab^*a} \text{ (Con.)}}}} \quad \frac{\overline{a} \text{ (Lt.)}}{\frac{\overline{b} \text{ (Lt.)}}{\frac{\overline{a^*} \text{ (Ét.)}}{\frac{\overline{ba^*} \text{ (Con.)}}{\overline{aba^*} \text{ (Con.)}}}} \\
 \hline
 \overline{ab^*a | aba^*} \text{ (Un.)}
 \end{array}$$

3. Dans cette question, il est attendu que soit donnés des détails de construction qui permettent au correcteur de s'assurer que la solution n'est pas sortie du chapeau. Des réponses qui se contentent de proposer un automate sans autre commentaire ne sont pas acceptées.

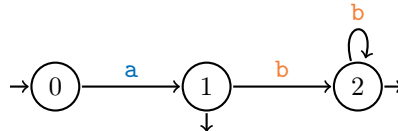
La branche de gauche de l'arbre de dérivation nous guide dans la construction de l'automate de Glushkov. Nous introduisons les symboles **a** et **b**.



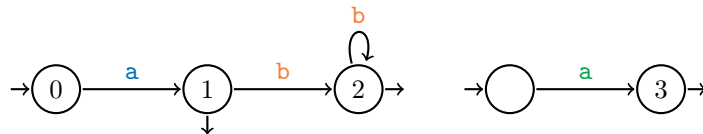
Nous construisons l'étoile de Kleene **b***.



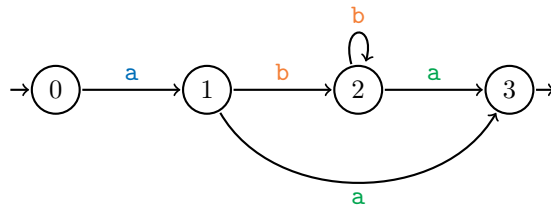
Nous concaténons pour obtenir l'automate de **ab***.



Nous introduisons les symboles **a**

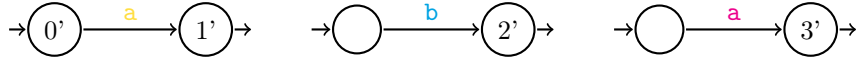


Enfin, nous concaténons pour obtenir l'automate de **ab*a**.

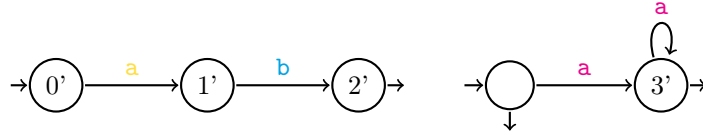


Nous faisons les vérifications qui s'imposent pour nous assurer que nous n'avons pas commis de faute de calcul : il y avait 3 lettres dans l'expression régulière r_1 et l'automate $\mathcal{A}_1$ contient bien $1 + 3$ états. Il n'y a pas de transition qui pointe vers l'état initial. Toutes les transitions qui pointent vers un état donné sont étiquetées par la même lettre. Par ailleurs, les mots **aa** et **aba** de la question 1 sont bien reconnus.

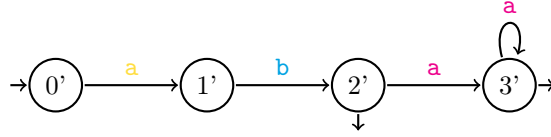
4. La branche de droite de l'arbre de dérivation nous guide dans la construction de l'automate de Glushkov. Nous introduisons les symboles **a**, **b** et **a**.



Nous concaténons pour obtenir l'automate de **ab** et appliquons la construction de l'étoile pour obtenir l'automate de **a***.



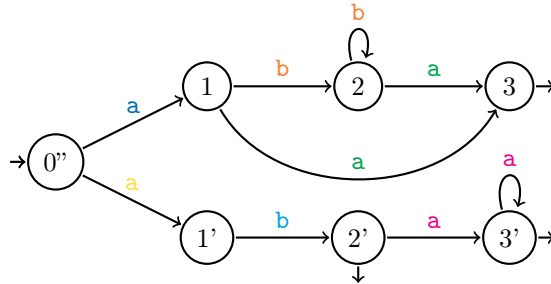
Avec une dernière concaténation, nous obtenons l'automate $\mathcal{A}_2$ de l'expression régulière **aba***.



Nous faisons les vérifications qui s'imposent : il y avait 3 lettres dans l'expression régulière r_2 et l'automate $\mathcal{A}_2$ contient bien $1 + 3$ états. Il n'y a pas de transition qui pointe vers l'état initial. Toutes les transitions qui pointent vers un état donné sont étiquetées par la même lettre avec la même couleur. Par ailleurs, les mots **ab** et **aba** de la question 1 sont bien reconnus.

5. Dans l'automate de Glushkov de l'expression régulière r , il y a un état de plus que de lettres dans r .
6. L'automate obtenu par simple juxtaposition de $\mathcal{A}_1$ et $\mathcal{A}_2$ ne peut pas correspondre à l'automate de l'expression régulière r : il possède un état de trop par rapport à la relation rappelée à la question 5.

En vérité, l'automate de Glushkov s'obtiendrait en fusionnant les deux états initiaux. Ici, on aurait donc, en créant un nouvel état initial, l'automate suivant :

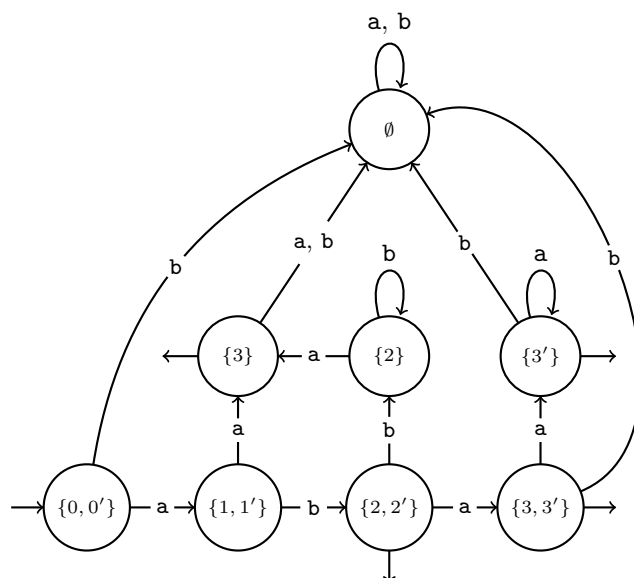


Remarque. Le mot « juxtaposer » a posé des soucis de compréhension à quelques copies : il avait le sens du dictionnaire qui est « Placer (une ou plusieurs choses) à côté (d'une ou plusieurs autres) »¹. La question a été corrigée avec souplesse par les professeurs.

7. L'état initial est l'ensemble $\{0, 0'\}$. Nous construisons au fur et à mesure la table des transitions et signalons les états finaux

État	Transition par a	Transition par b	Final
$\{0, 0'\}$	$\{1, 1'\}$	$\emptyset$	Non
$\{1, 1'\}$	$\{3\}$	$\{2, 2'\}$	Non
$\emptyset$	$\emptyset$	$\emptyset$	Non
$\{3\}$	$\emptyset$	$\emptyset$	Oui
$\{2, 2'\}$	$\{3, 3'\}$	$\{2\}$	Oui
$\{3, 3'\}$	$\{3'\}$	$\emptyset$	Oui
$\{2\}$	$\{3\}$	$\{2\}$	Non
$\{3'\}$	$\{3'\}$	$\emptyset$	Oui

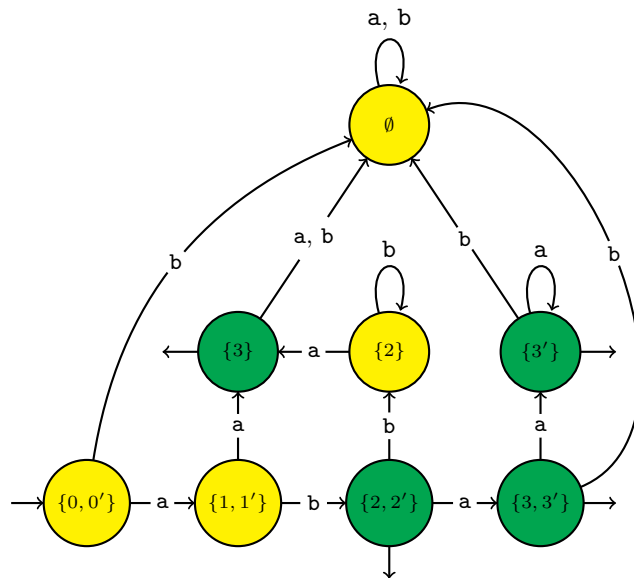
Nous pouvons également représenter cet automate.



8. L'automate $\mathcal{D}$ que nous venons de dessiner possède un puits qui est l'état $\emptyset$.
 9. Afin de minimiser l'automate $\mathcal{D}$, nous commençons par la partition la plus grossière qui soit et qui sépare les états finaux et non finaux :

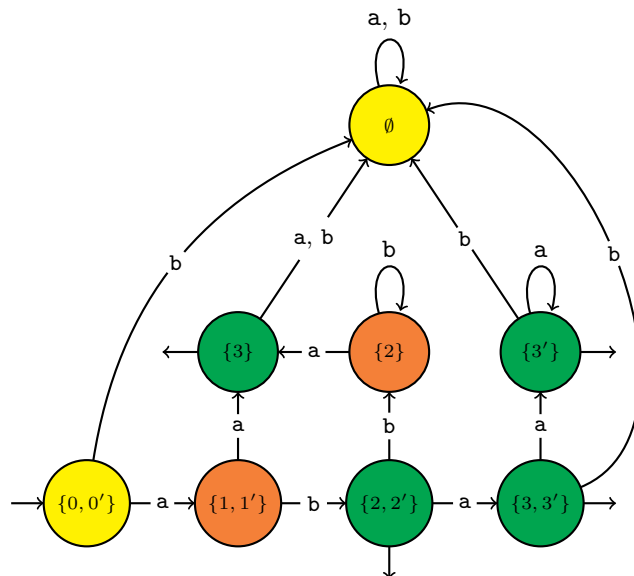
$\{0, 0'\}, \{1, 1'\}, \{2\}, \emptyset$	$\{3\}, \{3'\}, \{3, 3'\}, \{2, 2'\}$
--	---------------------------------------

1. Voir par exemple <https://www.cnrtl.fr/definition/juxtaposer>



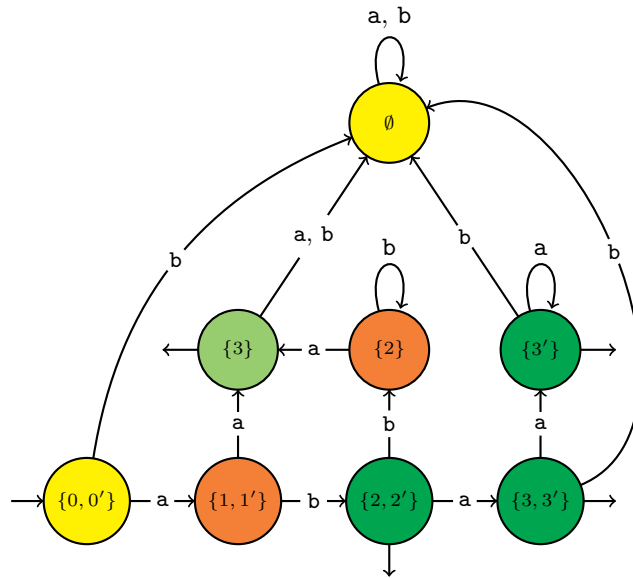
Nous affinons l'automate en divisant les états de couleur jaune en fonction de la transition d'étiquette **a**.

$\{0, 0'\}, \emptyset$	$\{1, 1'\}, \{2\}$	$\{3\}, \{3'\}, \{3, 3'\}, \{2, 2'\}$
------------------------	--------------------	---------------------------------------



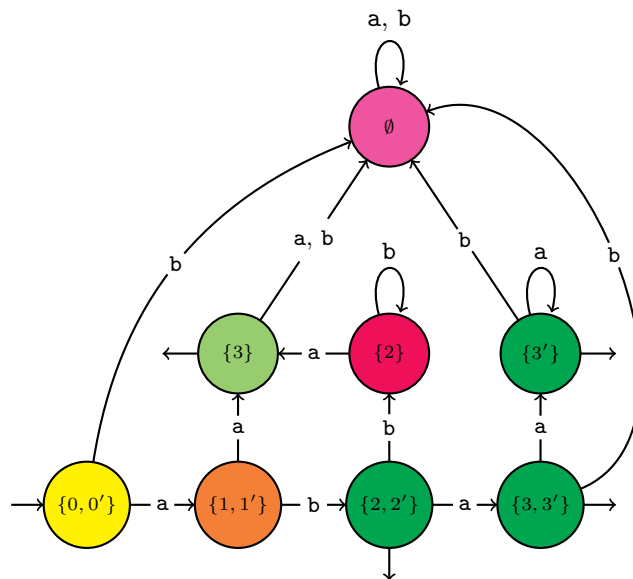
Nous séparons les états en vert en fonction de la transition d'étiquette **a**.

$\{0, 0'\}, \emptyset$	$\{1, 1'\}, \{2\}$	$\{3\}$	$\{3'\}, \{3, 3'\}, \{2, 2'\}$
------------------------	--------------------	---------	--------------------------------



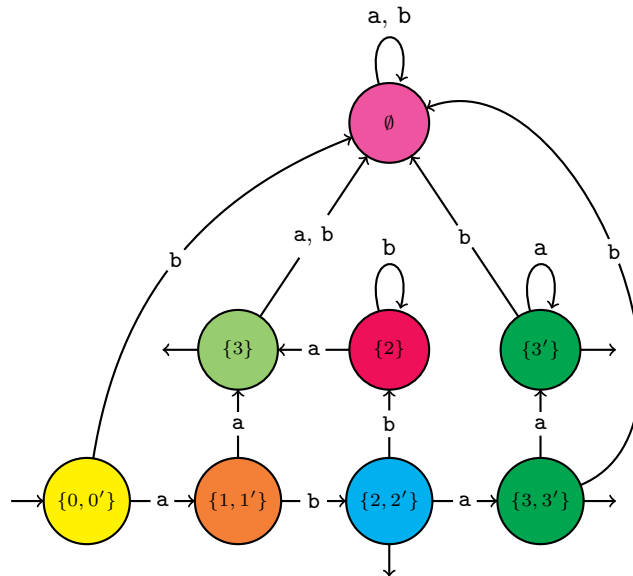
Nous séparons les états en jaune en fonction de la transition d'étiquette **b**.
 Nous séparons également les états en orange en fonction de la transition d'étiquette **b**.

$\{0, 0'\}$	$\emptyset$	$\{1, 1'\}$	$\{2\}$	$\{3\}$	$\{3'\}, \{3, 3'\}, \{2, 2'\}$
-------------	-------------	-------------	---------	---------	--------------------------------



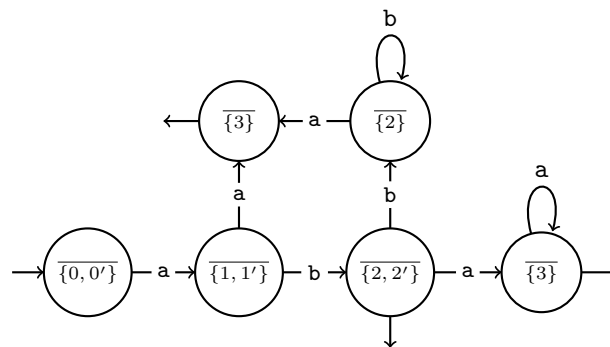
Nous séparons les états en vert en fonction de la transition d'étiquette **a**.

$\{0, 0'\}$	$\emptyset$	$\{1, 1'\}$	$\{2\}$	$\{3\}$	$\{3'\}, \{3, 3'\}$	$\{2, 2'\}$
-------------	-------------	-------------	---------	---------	---------------------	-------------



Nous réalisons qu'il est possible de fusionner les états $\{3'\}$ et $\{3, 3'\}$. Tous les autres états sont séparés.

10. L'automate minimal émondé est l'automate suivant



L'application de l'algorithme d'élimination des états fait exploser le nombre de symboles qui apparaissent. L'automate émondé compte déjà 8 transitions. Dans le cas présent, on obtiendrait une expression régulière terriblement grosse par rapport à l'expression régulière initiale, qui ne contient que 9 symboles (métacaractères compris), et qui serait très désagréable à lire.

Remarque : la minimisation de l'automate n'était pas attendue.

* *
*